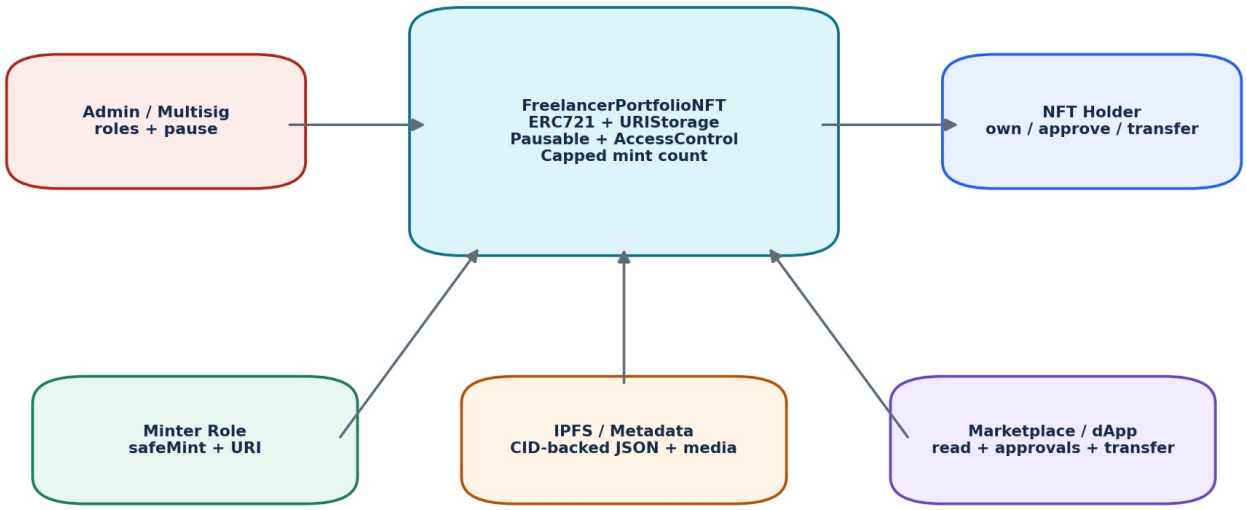
DAY 8

NFT Development: ERC-721, ERC-1155, Metadata & IPFS

Ownership, Token IDs, Approvals, Safe Transfers, Metadata, Minting, Multi-Tokens, Security, Testing & Client Delivery

60-MINUTE OUTCOME Students move from fungible ERC-20 accounting to unique token ownership, NFT metadata, marketplace approvals and multi-token design. The portfolio result is a role-controlled, capped, pausable ERC-721 collection.



Lecture	Duration	Portfolio Outcome
Day 8	60 minutes	FreelancerPortfolioNFT - ERC-721 collection with metadata
Level	Beginner	Assumes Days 5-7 Solidity + ERC-20 fundamentals
Career focus	LinkedIn + Upwork	NFT collection creation, debugging, metadata and marketplace integration

SAFETY / PRODUCT NOTE An NFT contract can control token ownership, but it does not automatically grant copyright, licensing rights, authenticity, legal title or permanent media hosting. Client requirements must state what the NFT actually represents.

1. Learning Objectives & Minute-by-Minute Schedule

By the end of Day 8, students should be able to trace an NFT from minting to ownership, metadata resolution, approval and transfer; compare ERC-721 with ERC-1155; and identify common production mistakes that create client support problems.

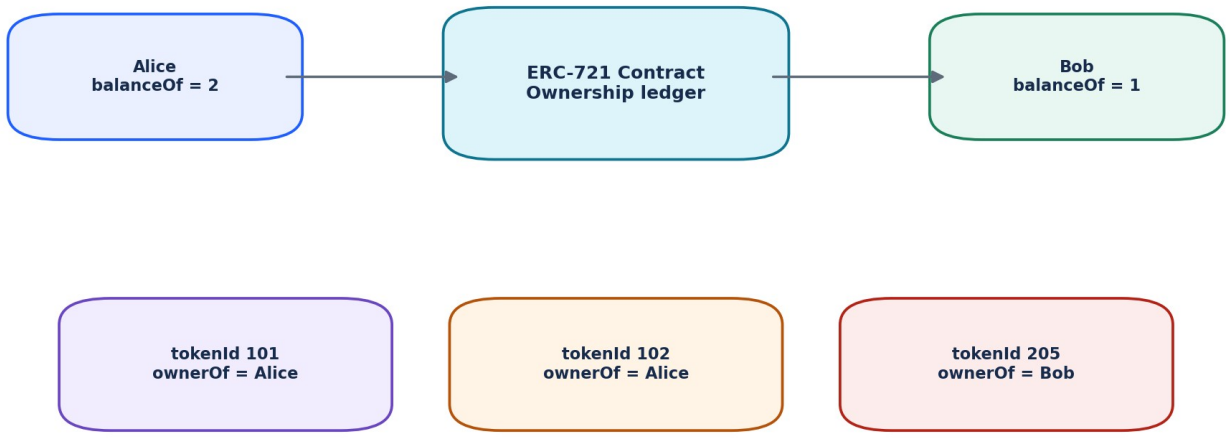
- Explain fungible vs non-fungible assets and the role of tokenId.
- Recognize the core ERC-721 ownership, transfer and approval functions and events.
- Explain transferFrom vs safeTransferFrom and IERC721Receiver.
- Explain tokenURI, metadata JSON, image/media URIs and common marketplace metadata conventions.
- Describe IPFS content addressing, CIDs, gateways and the difference between addressing and persistence.
- Use OpenZeppelin ERC721, URIStorage, Pausable and AccessControl components.
- Compare token-specific approval with operator approval for all NFTs in a collection.
- Explain ERC-1155 multi-token balances and batch transfers.
- Test minting, permissions, transfers, metadata, cap and paused-state behavior.

Minutes	Topic	Instructor Outcome
0-5	NFT mental model	Understand uniqueness, token IDs and ownership.
5-13	ERC-721 interface	Recognize ownership, transfer, approval and events.
13-20	Mint + burn + safe transfer	Trace token lifecycle and receiver checks.
20-29	Metadata + tokenURI + IPFS	Understand where metadata/media live and how clients resolve them.
29-36	Approvals + marketplace flow	Understand token approval vs operator approval.
36-43	OpenZeppelin extensions	Build maintainable NFT contracts from vetted components.
43-49	ERC-1155	Model multiple IDs, quantities and batch operations.
49-57	Hands-on portfolio NFT	Read, deploy and exercise one complete contract.
57-60	Testing + quiz + homework	Validate behavior and prepare portfolio work.

2. NFT Mental Model: Unique Token IDs | 0-5 min

ERC-20 asks: "How many units does this address own?" ERC-721 adds a second question: "Who owns this specific token ID?" Each NFT is identified by an integer tokenId inside a collection contract.

Concept	ERC-20	ERC-721
Primary identity	Amount of fungible units	Unique tokenId
Typical ownership read	balanceOf(address)	ownerOf(tokenId) + balanceOf(address)
Transfer input	recipient + amount	from + to + tokenId
Approval model	Allowance amount	Specific token approval or operator-for-all
Common use	Currencies, utility units	Collectibles, credentials, positions, tickets, assets



Unlike ERC-20 balances, each ERC-721 token ID maps to one current owner.

BEGINNER DISTINCTION The token ID is not the artwork. It is an identifier in the smart contract. Metadata referenced by tokenURI can describe the item and point to media.

3. Core ERC-721 Interface | 5-13 min

ERC-721 standardizes ownership and transfer behavior so wallets and marketplaces can interact with NFT collections through a common interface. ERC-165 interface detection is part of the standard design.

Member	Purpose	State?
balanceOf(owner)	Count NFTs owned by an address	Read
ownerOf(tokenId)	Return owner of one token ID	Read
transferFrom(from,to,id)	Transfer an NFT without receiver callback safety	Write
safeTransferFrom(...)	Transfer and check contract recipients	Write
approve(to, tokenId)	Approve one address for one token	Write
getApproved(tokenId)	Read approved address for one token	Read
setApprovalForAll(operator, bool)	Authorize/revoke operator across caller collection tokens	Write
isApprovedForAll(owner, operator)	Read operator authorization	Read
Transfer / Approval / ApprovalForAll	Standard logs	Event

```
interface IERC721Core {
    function balanceOf(address owner) external view returns (uint256);
    function ownerOf(uint256 tokenId) external view returns (address);
    function safeTransferFrom(address from, address to, uint256 tokenId) external;
    function transferFrom(address from, address to, uint256 tokenId) external;
    function approve(address to, uint256 tokenId) external;
    function getApproved(uint256 tokenId) external view returns (address);
    function setApprovalForAll(address operator, bool approved) external;
    function isApprovedForAll(address owner, address operator) external view returns (bool);
}
```

INTERVIEW POINT balanceOf(address) returns how many NFTs an owner has; ownerOf(tokenId) returns who owns one specific NFT.

4. Transfer Events & Ownership State

When token #101 moves from Alice to Bob, the collection updates its ownership state and emits Transfer(Alice, Bob, 101). Indexers, explorers and marketplaces watch these events to build searchable histories.

```
// Conceptual result of transferring token #101
before: ownerOf(101) == alice

safeTransferFrom(alice, bob, 101)

after:  ownerOf(101) == bob
        balanceOf(alice) decreases by 1
        balanceOf(bob) increases by 1

emit Transfer(alice, bob, 101);
```

Who is allowed to transfer?

- The token owner.
- The address specifically approved for that token ID.
- An operator approved through setApprovalForAll.
- Contract logic must still satisfy any additional restrictions introduced by the collection.

DEBUGGING HABIT When a transfer reverts, inspect current owner, caller, token-specific approval, operator approval, paused state, recipient type and transaction logs before guessing.

5. Minting, Burning & Token Lifecycle | 13-20 min

Minting creates an ownership record for a token ID that did not previously exist. Burning removes the active ownership record. In standard event semantics, minting emits Transfer from the zero address; burning emits Transfer to the zero address.

```
// OpenZeppelin-style minting
_safeMint(to, tokenId);

// If using ERC721URIStorage
_setTokenURI(tokenId, metadataURI);

// Burning is available through ERC721Burnable or internal _burn(tokenId).
```

Lifecycle action	Ownership meaning	Typical Transfer event
Mint	Create token ownership	0x0 -> recipient
Transfer	Move existing ownership	owner -> new owner
Burn	Remove active token ownership	owner -> 0x0

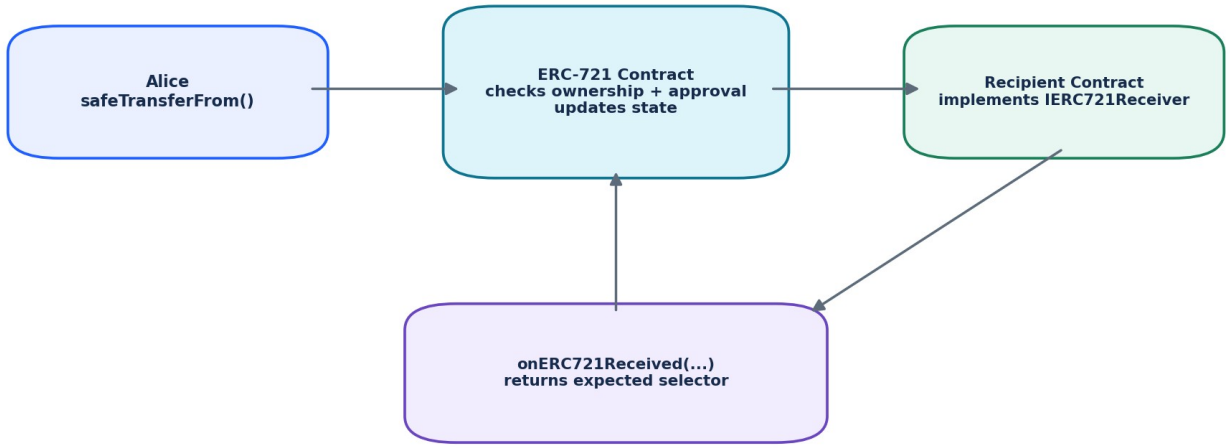
Token ID strategy

- Sequential IDs: 0, 1, 2, 3... easy to reason about and index.
- Externally meaningful IDs: useful when IDs map to records, but require collision/validation design.
- Hash-derived IDs: can encode deterministic uniqueness, but readability and collision assumptions must be considered.
- Never assume a client wants IDs to be reusable after burn; define the lifecycle explicitly.

ACCESS CONTROL Public minting, allowlist minting, role-based minting and paid minting are different products. The contract should implement the client's issuance policy explicitly.

6. Why safeTransferFrom Matters

An externally owned account can hold an NFT without implementing code. A smart-contract recipient is different: if it has no NFT-receiver logic, a raw transfer may leave the token inaccessible to the application. ERC-721 safe transfers call the receiver hook for contract recipients.



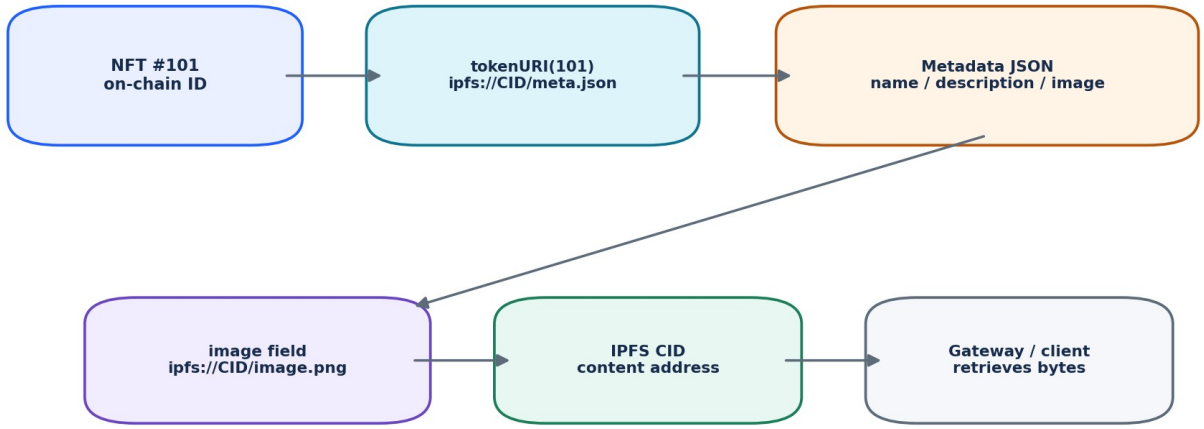
Safe transfer reduces the risk of NFTs being trapped in contracts that cannot receive them.

```
interface IERC721Receiver {
    function onERC721Received(
        address operator,
        address from,
        uint256 tokenId,
        bytes calldata data
    ) external returns (bytes4);
}
```

SECURITY NOTE The safe-transfer callback is an external call. When you add custom logic around minting or transfers, think about reentrancy and state ordering rather than assuming NFT code cannot re-enter.

7. NFT Metadata & tokenURI | 20-29 min

The ERC-721 metadata extension provides name(), symbol() and tokenURI(tokenId). tokenURI commonly returns a URI for a JSON metadata document. The metadata document can then reference image, animation or other media.



The NFT token is on-chain; metadata and media may be stored elsewhere and referenced by URI.

```
{
  "name": "Blockchain Portfolio #1",
  "description": "A demo NFT created during Day 8.",
  "image": "ipfs://bafy.../portfolio-1.png",
  "attributes": [
    {"trait_type": "Track", "value": "Blockchain"},
    {"trait_type": "Level", "value": "Beginner"}
  ]
}
```

STANDARD VS CONVENTION ERC-721 defines a metadata JSON schema with fields such as name, description and image. The attributes array is a widely used marketplace convention, not a core ERC-721 requirement.

8. On-Chain vs Off-Chain Metadata

Approach	Advantages	Tradeoffs
HTTP/HTTPS metadata	Simple web hosting and updates	Server/domain dependency; content may change or disappear
IPFS URI	Content-addressed and verifiable by CID	Availability still depends on someone providing/pinning content
Fully on-chain data URI	Maximum chain-level persistence/independence	High gas/storage complexity; media size limitations
Hybrid	Flexible balance of cost and permanence	More architecture and lifecycle decisions

IPFS: content addressing, not automatic permanence

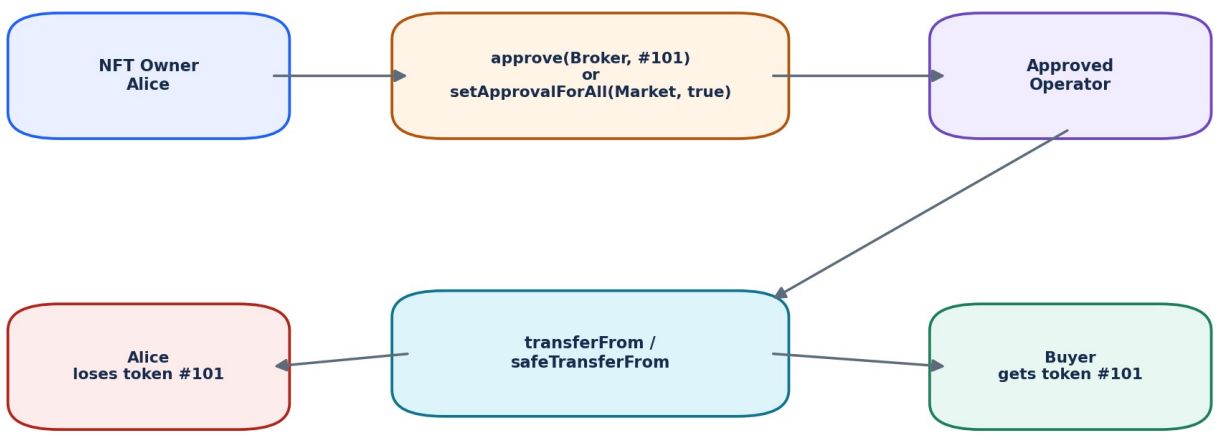
IPFS uses content identifiers (CIDs) derived from content. If the bytes change, the CID changes. A public gateway can translate an ipfs:// style reference into ordinary HTTP retrieval, but the gateway is not the ownership layer of the NFT.

IMPORTANT A CID lets clients ask for specific content and verify what they received. It does not guarantee that a provider will retain those bytes forever. Plan pinning/persistence and handoff responsibilities with the client.

Metadata mutability

- If tokenURI can be changed by an admin, the NFT metadata is mutable.
- If tokenURI points to mutable HTTP content, the displayed NFT can change without an on-chain URI change.
- If the product promises "frozen" metadata, define exactly how and when metadata becomes immutable.
- Document baseURI, reveal logic and administrative powers in the client handoff.

9. Approvals & Marketplace Flow | 29-36 min



Approval authorizes transfer; it does not itself change ownership.

Two approval scopes

Method	Scope	Typical use
approve(operator, tokenId)	One token ID	Delegate sale/transfer for a specific NFT
setApprovalForAll(operator, true)	All caller NFTs in this collection	Marketplace or portfolio operator integration

A marketplace often needs transfer authority so it can execute a sale after a buyer satisfies the order conditions. That authorization is separate from listing data, signatures and payment settlement.

PHISHING RISK setApprovalForAll is powerful. A malicious operator approved for a collection may transfer every NFT that the owner holds in that collection. Wallet UX and client support should treat operator approvals as security-sensitive.

10. Marketplace Transaction Mental Model

1. Seller owns NFT #101.

2. Seller signs or submits a listing/order according to the marketplace design.

3. Marketplace contract has the required token approval or receives it during the flow.

4. Buyer satisfies the order and payment conditions.

5. Marketplace transfers NFT #101 from seller to buyer.

6. Payment and platform/royalty logic execute according to marketplace rules.

7. Explorer logs show ownership transfer and payment events.

Royalties: communicate, not guarantee

ERC-2981 is a common royalty-information interface that can tell a marketplace who should receive a royalty and how much for a sale price. The NFT contract cannot force every external marketplace to honor that information unless the marketplace/payment architecture itself enforces it.

CLIENT QUESTION When a client says "10% royalties", ask: Which marketplaces? Is royalty information enough, or must transfer/payment be restricted through an enforcing marketplace? These are different designs.

11. OpenZeppelin ERC-721 Building Blocks | 36-43 min

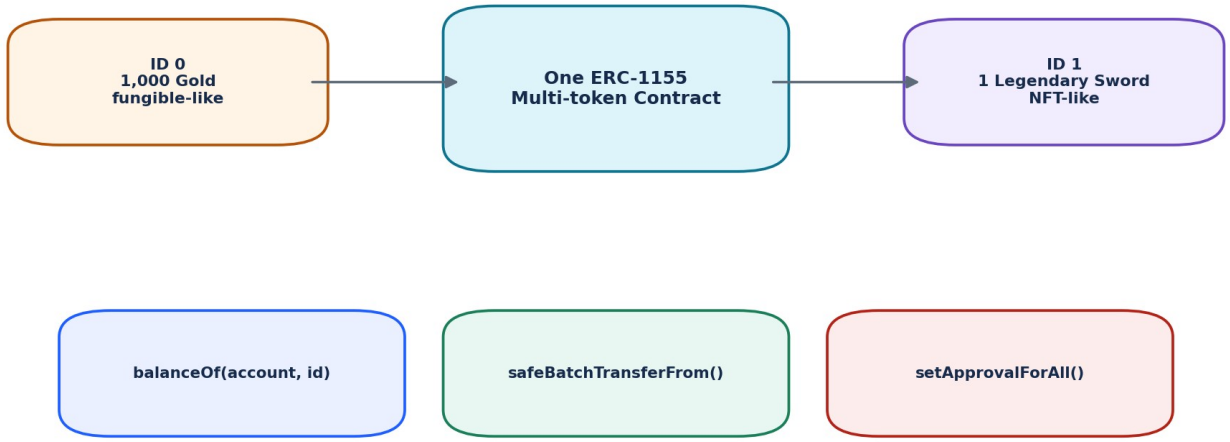
For client work, prefer standard, reviewed components over rewriting ownership, approval and transfer bookkeeping. OpenZeppelin Contracts 5.x provides ERC-721 implementations and extensions that can be combined through Solidity inheritance.

Component	What it adds	Use carefully because...
ERC721	Core ownership / transfer / approval behavior	You still define mint policy and metadata strategy
ERC721URIStorage	Per-token URI storage	Adds storage writes; decide whether metadata is mutable
ERC721Burnable	Holder/approved burn entry point	Burn policy must match product requirements
ERC721Pausable	Pause integration through transfer update path	Powerful admin control; disclose who can pause
ERC721Enumerable	On-chain token enumeration	Can add gas/storage overhead; indexers may be better
ERC721Royalty / ERC2981	Royalty information	Does not by itself enforce marketplace payment

OPENZEPPELIN 5.x Modern token customizations route through _update rather than the older before/after token-transfer hooks. When following tutorials, check the library major version before copying override code.

12. ERC-1155 Multi-Token Standard | 43-49 min

ERC-1155 lets one contract represent many token IDs with quantities. A token ID can behave like a fungible game currency, a one-of-one item, or a limited-edition item depending on how many units exist.



ERC-1155 can represent many token IDs and quantities in one contract, with batch operations.

Concept	ERC-721	ERC-1155
Ownership model	One owner per token ID	Balance per account per token ID
Quantity per ID	Effectively 1 active owner unit	Can be 1 or many units
Batch transfers	Not core	Core safeBatchTransferFrom
Approval	Specific token + operator-for-all	Operator-for-all in core standard
Good fit	Unique collections / deeds	Games, editions, inventories, mixed asset sets

```
// Mental model for ERC-1155
balanceOf(alice, GOLD_ID)  -> 5000
balanceOf(alice, SWORD_ID) -> 1

safeBatchTransferFrom(
    alice,
    bob,
    [GOLD_ID, SWORD_ID],
    [250, 1],
    ""
);
```

13. Choosing ERC-721 vs ERC-1155

Client requirement	Better starting point	Reason
10,000 unique profile NFTs	ERC-721	Simple unique ownership model and broad marketplace support
Game with gold, potions, unique swords	ERC-1155	Many IDs and quantities in one contract
Event tickets with one unique seat per token	ERC-721	Each seat/ticket is individually identifiable
100 copies of the same membership edition	ERC-1155	One ID can have quantity 100
Mix of fungible and NFT-like in-game assets	ERC-1155	Fungibility-agnostic multi-token model

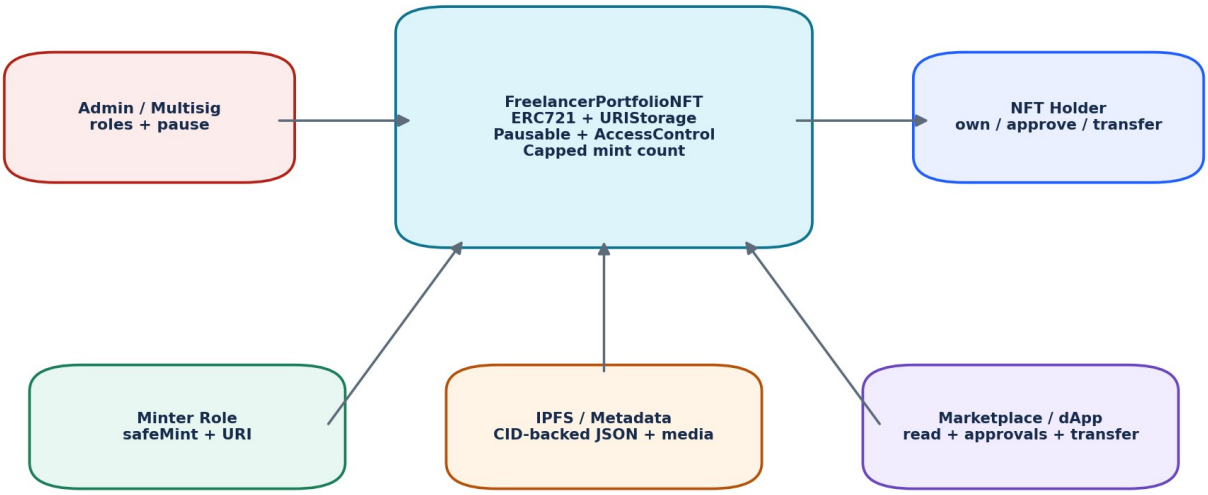
ARCHITECTURE RULE Choose a token standard from the ownership and transfer semantics the product needs - not because one standard sounds newer or "more advanced."

When not to tokenize

If a client only needs a database record, access control, file ownership or an internal certificate, blockchain may add cost and complexity without a clear benefit. A professional freelancer should be able to recommend not using an NFT when requirements do not need public verifiability, transferability or on-chain ownership.

14. Hands-On Project: FreelancerPortfolioNFT | 49-57 min

The practical contract uses OpenZeppelin Contracts 5.x. It keeps the design intentionally small: role-controlled minting, a maximum number of lifetime mints, per-token metadata URIs and an emergency pause. Burn is omitted so the lifecycle stays simple for this beginner project.




```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC721} from "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import {ERC721URIStorage} from "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";
import {ERC721Pausable} from "@openzeppelin/contracts/token/ERC721/extensions/ERC721Pausable.sol";
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";

contract FreelancerPortfolioNFT is
    ERC721,
    ERC721URIStorage,
    ERC721Pausable,
    AccessControl
{
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");

    uint256 public immutable maxSupply;
    uint256 private _nextTokenId;

    error MaxSupplyReached();

    constructor(address admin, uint256 maxSupply_)
        ERC721("Freelancer Portfolio", "FPORT")
    {
        require(admin != address(0), "zero admin");
        require(maxSupply_ > 0, "zero supply");
        maxSupply = maxSupply_;
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(MINTER_ROLE, admin);
    }
```

15. FreelancerPortfolioNFT - Mint, Pause & Overrides

```
    function safeMint(address to, string calldata uri)
        external
        onlyRole(MINTER_ROLE)
        returns (uint256 tokenId)
    {
        tokenId = _nextTokenId;
        if (tokenId >= maxSupply) revert MaxSupplyReached();

        _nextTokenId = tokenId + 1;
        _safeMint(to, tokenId);
        _setTokenURI(tokenId, uri);
    }

    function totalMinted() external view returns (uint256) {
        return _nextTokenId;
    }

    function pause() external onlyRole(DEFAULT_ADMIN_ROLE) {
        _pause();
    }

    function unpause() external onlyRole(DEFAULT_ADMIN_ROLE) {
        _unpause();
    }

    function tokenURI(uint256 tokenId)
        public
        view
        override(ERC721, ERC721URIStorage)
        returns (string memory)
    {
        return super.tokenURI(tokenId);
    }

    function _update(address to, uint256 tokenId, address auth)
        internal
        override(ERC721, ERC721Pausable)
        returns (address)
    {
        return super._update(to, tokenId, auth);
    }

    function supportsInterface(bytes4 interfaceId)
        public
        view
        override(ERC721, ERC721URIStorage, AccessControl)
        returns (bool)
    {
        return super.supportsInterface(interfaceId);
    }
}
```

DESIGN NOTE maxSupply here limits lifetime token IDs minted, not current circulating supply. Because this beginner contract has no burn function, the two values cannot diverge. If you add burn later, define whether burning should free mint capacity.

16. Hands-On Lab: Deploy & Exercise the NFT

Remix / local-test procedure

1. Create FreelancerPortfolioNFT.sol and install/import OpenZeppelin Contracts compatible with the code.

2. Compile with Solidity 0.8.24 or a compatible 0.8.x compiler required by your dependencies.

3. Deploy with your test account as admin and maxSupply = 5.

4. Call name(), symbol(), maxSupply() and totalMinted().

5. Mint token #0 to Account A with a test URI such as ipfs://exampleCID/0.json.

6. Confirm ownerOf(0), balanceOf(Account A), tokenURI(0) and totalMinted().

7. From Account A, approve Account B for token #0; confirm getApproved(0).

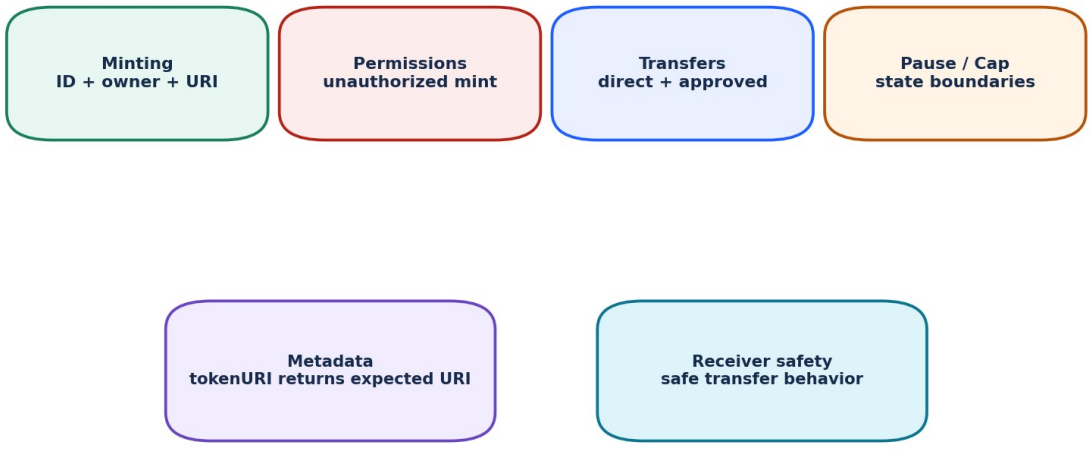
8. Transfer token #0 to Account B and confirm the new owner.

9. Pause as admin and prove that transfer/mint paths governed by ERC721Pausable revert.

10. Unpause and confirm normal behavior resumes.

USE TEST DATA Do not paste real seed phrases or production private keys into Remix, screenshots, homework, GitHub or client chat. Use disposable test accounts and testnet/local assets.

17. Testing Matrix - What a Client-Ready NFT Must Prove



An NFT project is not complete when mint() works; ownership, approvals, metadata and receiver safety must be tested.

Test	Expected result
Authorized mint	New token ID belongs to recipient and URI is stored
Unauthorized mint	Reverts for caller without MINTER_ROLE
Cap boundary	Mint beyond maxSupply reverts
Direct owner transfer	Owner changes and balances update
Specific approval	Approved address can transfer specified token
Operator approval	Approved operator can transfer owner tokens
Unauthorized transfer	Reverts
Paused state	Transfer/mint update path reverts
Unpause	Normal actions resume
Metadata read	tokenURI returns the expected URI
Safe recipient	Contract receiver must satisfy ERC721 receiver rules

Useful invariant ideas

- Every existing ERC-721 token ID has exactly one current owner.
- No account balance can become inconsistent with the set of token IDs it owns.
- An unapproved third party cannot transfer someone else's NFT.
- totalMinted never exceeds maxSupply.

18. Common NFT Security & Product Mistakes

Mistake	Why it hurts	Better practice
Unrestricted mint	Anyone can issue client assets	Explicit public/allowlist/role mint policy
Unlimited admin powers	Admin can surprise holders	Minimize, multisig, timelock or disclose powers
Mutable metadata undocumented	Displayed asset can change unexpectedly	Document freeze/update rules
HTTP server only	Broken domain can break NFT display	Use durable hosting/content addressing where appropriate
"IPFS = permanent" assumption	CID may become unavailable if nobody retains data	Plan persistence/pinning
Unsafe contract transfer	NFT may be sent to a contract not designed to receive it	Prefer safe transfer patterns
Approval phishing ignored	Operator may move all collection NFTs	Educate/review approval UX
Old tutorial overrides	Library major-version mismatch breaks compile/security assumptions	Use current docs for installed version
Royalties described as guaranteed	External marketplaces may not pay them	Clarify information vs enforcement
NFT = copyright assumption	Token ownership and IP rights are separate concepts	Define license/legal rights explicitly

19. Freelancer Debugging Scenarios

Scenario A - "My NFT minted, but the marketplace shows a blank image"

1. Read tokenURI(tokenId) directly.

2. Retrieve the metadata document.

3. Validate JSON syntax and expected name/image fields.

4. Resolve the image URI separately.

5. Check IPFS CID availability/pinning or HTTP status.

6. Check whether the marketplace caches metadata and needs refresh/indexing.

Scenario B - "Marketplace cannot transfer my NFT"

1. Verify ownerOf(tokenId).

2. Check getApproved(tokenId).

3. Check isApprovedForAll(owner, marketplaceOperator).

4. Check paused/restricted transfer logic.

5. Check whether the marketplace is calling the expected collection/network.

Scenario C - "safeTransferFrom to our vault contract reverts"

Inspect the vault contract. If it is the recipient, it must correctly implement IERC721Receiver and return the expected selector for safe ERC-721 reception.

20. Upwork / LinkedIn Client Requirement Translation

Client says...	Technical questions you should ask
"Create 10,000 NFTs"	ERC-721 or ERC-1155? fixed cap? public or admin mint? sequential IDs?
"Store images on IPFS"	Who uploads/pins? metadata CID too? freeze policy? backup/persistence?
"Add royalties"	ERC-2981 info only or enforced marketplace? percentage? recipient update rights?
"Users can mint"	Free/paid? per-wallet limit? allowlist? start/end time? withdraw destination?
"Make metadata editable"	Who can edit? until when? event/audit trail? holder expectations?
"Marketplace compatible"	Which marketplaces/chains? required interfaces? operator flow? metadata conventions?
"Admin can pause"	Which operations pause? who holds role? multisig? emergency process?

FREELANCER VALUE Strong developers turn vague NFT requests into explicit ownership, issuance, metadata, permissions, storage and marketplace requirements before writing code.

21. Interview Questions + 10-Question Quiz

Interview questions

- What is the difference between ERC-20 balance accounting and ERC-721 ownership?
- Why does ERC-721 have both ownerOf and balanceOf?
- What is the difference between approve and setApprovalForAll?
- Why is safeTransferFrom generally preferred for contract recipients?
- What does tokenURI return?
- What does an IPFS CID identify?
- Why does IPFS not automatically guarantee permanent availability?
- When would you choose ERC-1155 instead of ERC-721?
- How would you pause an OpenZeppelin 5.x ERC-721 collection?
- Does ERC-2981 force a marketplace to pay royalties?

Quick quiz

1. Which function tells you who owns token #42?

2. Which event records an ERC-721 ownership move?

3. What approval gives one operator authority across an owner's collection tokens?

4. What callback is checked for safe transfer to a contract?

5. Does tokenURI have to contain the image bytes directly?

6. What changes when IPFS file bytes change?

7. Can ERC-1155 hold multiple token IDs in one contract?

8. What should happen when an unauthorized account calls role-controlled mint?

9. Why should metadata mutability be documented?

10. What is one reason old OpenZeppelin tutorials may fail with 5.x?

ANSWERS 1 ownerOf. 2 Transfer. 3 setApprovalForAll. 4 onERC721Received. 5 No. 6 The CID changes. 7 Yes. 8 Revert. 9 Holders/clients need to know whether content can change. 10 Token customization hooks/API changed across major versions; use current-version docs.

22. Homework & Portfolio Deliverable

Required homework

1. Deploy FreelancerPortfolioNFT on a local chain or testnet using only test accounts.

2. Create two metadata JSON files and two image files. Record the intended URI structure.

3. Mint two NFTs to two different test accounts.

4. Capture ownerOf, tokenURI, Transfer event and approval evidence.

5. Demonstrate one token-specific approval and one operator approval in a safe test environment.

6. Demonstrate pause -> expected revert -> unpause.

7. Write a one-page README explaining collection purpose, roles, metadata strategy and security assumptions.

8. Write five tests from the testing matrix before Day 9.

Portfolio README structure

```
# Freelancer Portfolio NFT

## Features
- ERC-721 ownership and safe transfers
- Role-controlled minting
- Maximum lifetime mint count
- Per-token metadata URI
- Emergency pause

## Architecture
Wallet -> NFT Contract -> tokenURI -> Metadata -> Media

## Roles
DEFAULT_ADMIN_ROLE: role administration + pause
MINTER_ROLE: mint new NFTs

## Security Notes
- Testnet/local demo only
- Metadata persistence must be managed separately
- Admin role should use stronger custody for production

## Deployment
Network: <testnet/local>
Contract: <address>
OpenZeppelin version: <version>
```

23. Day 8 Cheat Sheet

Concept	Remember
ERC-721	Standard interface for non-fungible token ownership and transfers
tokenId	Unique identifier inside a collection contract
ownerOf(id)	Current owner of one NFT
balanceOf(owner)	How many collection NFTs an address owns
approve	Authorize one address for one token ID
setApprovalForAll	Authorize an operator across caller's tokens in the collection
safeTransferFrom	Transfer with receiver check for contract recipients
tokenURI	URI to metadata for a token ID
Metadata	Descriptive JSON; commonly references image/media
CID	IPFS content identifier derived from content
ERC-1155	Multi-token standard with balances per account per ID and batch operations
ERC-2981	Royalty information interface; not universal enforcement
OZ 5.x customization	Token transfer/mint/burn custom logic commonly routes through _update
Client delivery	Document roles, mint rules, metadata persistence, upgrade/freeze policy and tests

60-second architecture recall

```
User / Marketplace
      |
      v
ERC-721 Collection
  | ownerOf / approvals / transfers
  | tokenURI(tokenId)
      v
Metadata JSON
  | image / media URI
      v
IPFS / HTTP / on-chain content
```

READY FOR DAY 9 Day 9 moves from building contracts to proving them: unit tests, revert tests, event tests, fuzz testing, edge cases and test-driven smart-contract debugging.

24. Instructor Notes & Primary References

Instructor whiteboard sequence

1. Draw ERC-20 as address -> amount, then ERC-721 as tokenId -> owner.

2. Write ownerOf(101) and balanceOf(Alice); ask students what each answers.

3. Draw approve(one token) vs setApprovalForAll(collection operator).

4. Draw safeTransferFrom calling a receiver contract callback.

5. Draw tokenId -> tokenURI -> metadata JSON -> image URI -> IPFS CID.

6. Draw ERC-1155 as account + tokenId -> quantity.

7. Finish with FreelancerPortfolioNFT roles, cap, metadata and pause test plan.

Primary technical references

- EIP-721 - Non-Fungible Token Standard: https://eips.ethereum.org/EIPS/eip-721
- EIP-1155 - Multi Token Standard: https://eips.ethereum.org/EIPS/eip-1155
- OpenZeppelin Contracts 5.x - ERC-721: https://docs.openzeppelin.com/contracts/5.x/erc721
- OpenZeppelin Contracts 5.x - ERC-1155: https://docs.openzeppelin.com/contracts/5.x/erc1155
- OpenZeppelin Contracts 5.x - Token APIs: https://docs.openzeppelin.com/contracts/5.x/api/token/erc721
- EIP-2981 - NFT Royalty Standard: https://eips.ethereum.org/EIPS/eip-2981
- IPFS Docs - Content addressing / CIDs: https://docs.ipfs.tech/concepts/content-addressing/
- IPFS Docs - Gateways: https://docs.ipfs.tech/concepts/ipfs-gateway/

TEACHING PRIORITY Do not spend the hour on NFT market speculation. The professional learning target is ownership semantics, interoperable interfaces, metadata architecture, permissions, safe transfer behavior and client-ready testing.